Effects in parallel, with concurrent monads

Tarmo Uustalu
based on joint work with Bjarki Gunnarsson, Exequiel Rivas

ACT '26, Tallinn, 6–10 July 2026

Concurrency and effects

- In algebraic semantics of programming, *Kleene algebras* axiomatize sequential composition together with nondeterministic choice and finite iteration (for linear time).
- *Concurrent Kleene algebras* (Hoare et al. 2009/11) add parallel composition.
- In categorical semantics of effectful programming, *monads* (via their Kleisli categories) axiomatize sequential composition of effectful functions.
- but they do not axiomatize parallel composition.
- Could we add parallel composition, taking inspiration from concurrent Kleene algebras?
- Spoiler: Yes, with *concurrent monads* (Rivas, Jaskelioff 2019).

Concurrent monoids

(Gischer 1984/88; Hoare et al. 2009/11)

- A *concurrent monoid* is an ordered set $(M, \leq)$ carrying two ordered monoid structures $(\mathsf{id}, (;))$ and $(\mathsf{jd}, \|)$ related by interchange inequations

$$\mathsf{id} \leq \mathsf{jd}$$
$$\mathsf{jd} \,;\, \mathsf{jd} \leq \mathsf{jd}$$
$$\mathsf{id} \leq \mathsf{id} \,\|\, \mathsf{id}$$
$$(k \,\|\, \ell) \,;\, (m \,\|\, n) \leq (k \,;\, m) \,\|\, (\ell \,;\, n)$$

- If the 1st inequation holds as an equality, the concurrent monoid is said to be *normal*.

- $(;)$ is for sequential composition (in diagrammatic order), $\|$ is for parallel composition.

- $k \leq \ell$ typically means ℓ can behave in more ways than k.

- There are arguments for taking the converse order $\geq$ as basic instead of $\leq$. We follow the concurrent Kleene algebra tradition.

Concurrent monoids ctd

- The 4th inequation implies the 1st.
  So the 1st is redundant.

- The 1st inequation implies the converses of the 2nd and 3rd inequations.
  So the 2nd and 3rd actually hold as equalities for any concurrent monoid.

- If a concurrent monoid is normal (the 1st inequation holds as an equality), then $k \,;\, \ell \leq k \parallel \ell$ and $\ell \,;\, k \leq k \parallel \ell$.

- If $\leq$ is $=$, a concurrent monoid reduces to a *duoid*, which is by Eckmann–Hilton the same as a commutative monoid.

- Classically, one requires commutativity of $\parallel$ and normality. We don't.

Example: Idempotent semirings

- Any idempotent semiring $(S, 0, +, 1, \times)$ comes with an order $\leq$ defined by $m \leq n$ iff $m + n = n$.
- It induces two concurrent monoids, $(S, \leq, 0, +, 1, \times)$ and $(S, \geq, 1, \times, 0, +)$.

  In the first, $(;)$ is commutative and idempotent.

  In the second, $\|$ is commutative and idempotent.

- E.g., $(\mathbb{N}, \leq, 0, \max, 0, +)$ and $(\mathbb{N}, \geq, 0, +, 0, \max)$.

  These concurrent monoids are normal.

- E.g., the Boolean semiring $(\mathbb{B}, \supset, \mathsf{ff}, \vee, \mathsf{tt}, \wedge)$.

  This concurrent monoid is not normal.

Example: A 3-valued logic

- $(\{\mathsf{ff}, \mathsf{u}, \mathsf{tt}\}, \supset, \mathsf{ff}, \vee, \mathsf{tt}, \wedge)$ is a concurrent monoid where

  $\supset$ is generated by $\mathsf{ff} \supset m$, $m \supset \mathsf{tt}$ and

  $\vee$ and $\wedge$ are defined by

$\vee$	ff	u	tt
ff	ff	u	tt
u	u	tt	tt
tt	tt	tt	tt

$\wedge$	ff	u	tt
ff	ff	ff	ff
u	ff	ff	u
tt	ff	u	tt

  Here, both $(;)$ and $\|$ are nonidempotent.

Example: A 4-valued logic

- $(\{\mathsf{ff}, \mathsf{u_L}, \mathsf{u_R}, \mathsf{tt}\}, \supset, \mathsf{ff}, \vee, \mathsf{tt}, \wedge)$ is a concurrent monoid where

  $\supset$ is generated by $\mathsf{ff} \supset m$, $m \supset \mathsf{tt}$, $\mathsf{u_L} \supset \mathsf{u_R}$ and

  $\vee$ and $\wedge$ are defined by

$\vee$	ff	$\mathsf{u_L}$	$\mathsf{u_R}$	tt
ff	ff	$\mathsf{u_L}$	$\mathsf{u_R}$	tt
$\mathsf{u_L}$	$\mathsf{u_L}$	$\mathsf{u_L}$	$\mathsf{u_R}$	tt
$\mathsf{u_R}$	$\mathsf{u_R}$	tt	tt	tt
tt	tt	tt	tt	tt

$\wedge$	ff	$\mathsf{u_L}$	$\mathsf{u_R}$	tt
ff	ff	ff	ff	ff
$\mathsf{u_L}$	ff	ff	$\mathsf{u_L}$	$\mathsf{u_L}$
$\mathsf{u_R}$	ff	ff	$\mathsf{u_R}$	$\mathsf{u_R}$
tt	ff	$\mathsf{u_L}$	$\mathsf{u_R}$	tt

Here, both $(;)$ and $\|$ are both nonidempotent as well as noncommutative.

Example: Finite (un)weighted word languages

- Given a set Σ (an alphabet).

  $(\mathcal{M}_{\mathrm{f}}(\mathrm{List}\Sigma), \subseteq)$ (finited weighted languages of words) carries a concurrent monoid structure $(1, \cdot, 1, \sqcup\!\sqcup)$ where

  - $1 = \{[]\}$,
  - $L \cdot L' = \{u +\!\!+ v \mid u \in L, v \in L'\}$ (concatenation),
  - $L \sqcup\!\sqcup L' = \bigcup\{u \sqcup\!\sqcup v \mid u \in L, v \in L'\}$ (shuffle).

- $\mathcal{M}_{\mathrm{f}}$ can be replaced with $\mathcal{P}_{\mathrm{f}}$ (for unweighted languages) or the free S-semimodule monad (for S-weighted languages).

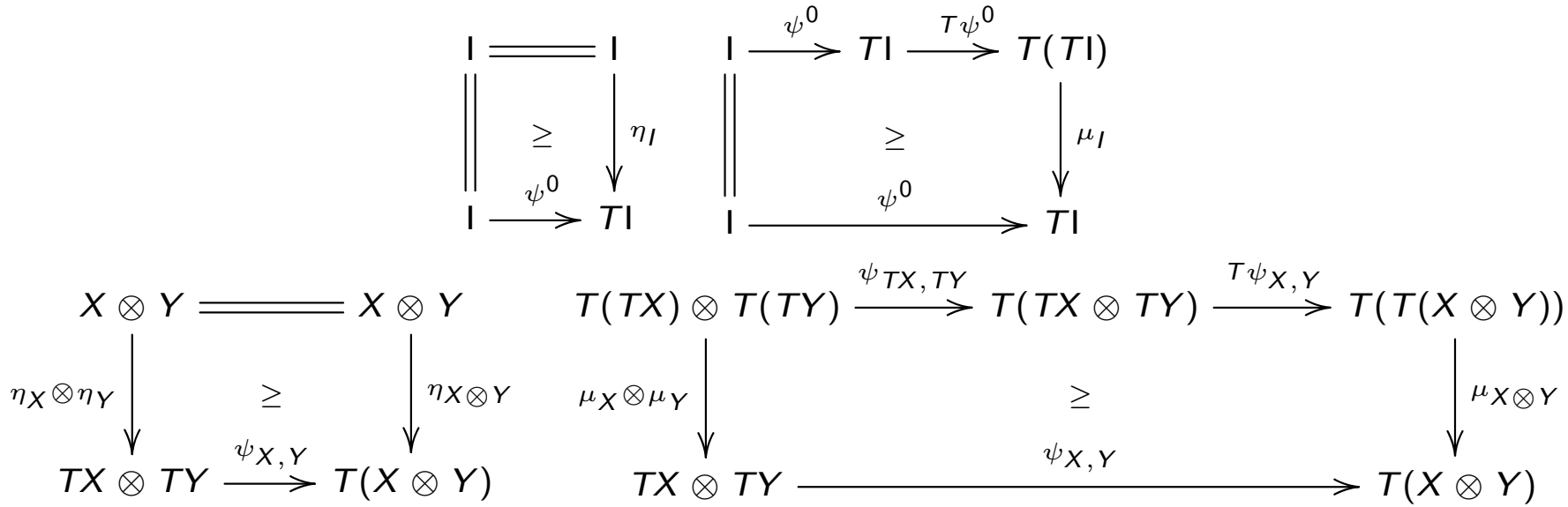
- Words can be replaced with Mazurkiewicz traces, labelled posets.

Concurrent monads

(Rivas, Jaskelioff 2019)

- A *concurrent monad* on an ordered monoidal category $(\mathbb{C}, \leq, \mathsf{I}, \otimes)$ is an ordered endofunctor T carrying both
  - an ordered monad structure (η, μ) and
  - an ordered lax monoidal functor structure (ψ^0, ψ)

  satisfying interchange inequations

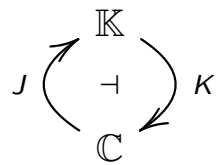


- If $\leq$ is $=$, this reduces to a *lax monoidal monad*, which is the same as a *commutative monad*.

Kleisli for a concurrent monad

- Given a concurrent monad $(T, \eta, \mu, \psi^0, \psi)$ on an ordered monoidal category $(\mathbb{C}, \leq, \mathsf{I}, \otimes)$

- Recall that the Kleisli ordered adjunction



for (T, η, μ) is given by these data:

- $|\mathbb{K}| = |\mathbb{C}|$, $\mathbb{K}(X, Y) = \mathbb{C}(X, TY)$,
- $k \leq^{\mathsf{K}} \ell$ in $\mathbb{K}(X, Y)$ iff $k \leq \ell$ in $\mathbb{C}(X, TY)$,
- $\mathrm{id}^{\mathsf{K}}{}_X = X \xrightarrow{\eta X} TX$,

  $k \,;^{\mathsf{K}}\, \ell = X \xrightarrow{k} TY \xrightarrow{T\ell} T(TZ) \xrightarrow{\mu Z} TZ$

  for $k \in \mathbb{K}(X, Y)$, $\ell \in \mathbb{K}(Y, Z)$,
- $JX = X$, $Jf = X \xrightarrow{f} Y \xrightarrow{\eta Y} TY$ for $f \in \mathbb{C}(X, Y)$,
- $KX = TX$, $Kk = TX \xrightarrow{Tk} T(TY) \xrightarrow{\mu Y} TY$ for $k \in \mathbb{K}(X, Y)$.

Kleisli for a concurrent monad ctd

- In addition, ψ^0 and ψ induce these data:
  - $\mathsf{jd} = \mathsf{I} \xrightarrow{\psi^0} T\mathsf{I}$

    $k \parallel \ell = X \otimes U \xrightarrow{k \otimes \ell} TY \otimes TV \xrightarrow{\psi_{Y,V}} T(Y \otimes V)$

    for $k \in \mathbb{K}(X, Y)$, $\ell \in \mathbb{K}(U, V)$.
- They satisfy these (in)equations:

$$(\mathsf{jd} \parallel k) \mathbin{;^{\mathsf{K}}} J\lambda_Y = J\lambda_X \mathbin{;^{\mathsf{K}}} k$$
$$k \mathbin{;^{\mathsf{K}}} J\rho_Y = J\rho_X \mathbin{;^{\mathsf{K}}} (k \parallel \mathsf{id})$$
$$((k \parallel \ell) \parallel m) \mathbin{;^{\mathsf{K}}} J\alpha_{Y,U,W} = J\alpha_{X,Z,V} \mathbin{;^{\mathsf{K}}} (k \parallel (\ell \parallel m))$$

$$\mathsf{id}^{\mathsf{K}}{}_{\mathsf{I}} \leq^{\mathsf{K}} \mathsf{jd}$$
$$\mathsf{jd} \mathbin{;^{\mathsf{K}}} \mathsf{jd} \leq^{\mathsf{K}} \mathsf{jd}$$
$$\mathsf{id}^{\mathsf{K}}{}_{X \otimes Y} \leq^{\mathsf{K}} \mathsf{id}^{\mathsf{K}}{}_X \parallel \mathsf{id}^{\mathsf{K}}{}_Y$$
$$(k \parallel \ell) \mathbin{;^{\mathsf{K}}} (m \parallel n) \leq^{\mathsf{K}} (k \mathbin{;^{\mathsf{K}}} m) \parallel (\ell \mathbin{;^{\mathsf{K}}} n)$$

$$J(f \otimes g) \mathbin{;^{\mathsf{K}}} (m \parallel n) = (Jf \mathbin{;^{\mathsf{K}}} m) \parallel (Jg \mathbin{;^{\mathsf{K}}} n)$$
$$(k \parallel \ell) \mathbin{;^{\mathsf{K}}} J(h \otimes i) = (k \mathbin{;^{\mathsf{K}}} Jh) \parallel (\ell \mathbin{;^{\mathsf{K}}} Ji)$$

- We could say $\mathbb{K}$ is an ordered lax-functorially monoidal category in that $\mathsf{I}^{\mathsf{K}} = (\mathsf{I}, \mathsf{jd})$, $\otimes^{\mathsf{K}} = (\otimes, \parallel)$ are lax-functors satisfying unitality and associativity.

Kleisli for a concurrent monad ctd

- (J, ι^0, ι) where $\iota^0 = \mathsf{id}^{\mathsf{K}}{}_I$ and $\iota_{X,Y} = \mathsf{id}^{\mathsf{K}}{}_{X \otimes Y}$ is an ordered oplax-naturally strict-monoidal functor
in that ι^0 and ι are oplax-natural transformations.

- (K, κ^0, κ) where $\kappa^0 = \psi^0$ and $\kappa_{X,Y} = \psi_{X,Y}$ is an ordered oplax-naturally lax-monoidal functor
in that κ^0 and κ are oplax-natural transformations.

- The unit η of the adjunction is oplax-monoidal natural.

- The counit ε of the adjunction where $\varepsilon_X = \mathsf{id}_{TX}$ is monoidal natural.

Example: Writer

- We consider examples on the ordered Cartesian monoidal closed category $(\mathrm{Poset}, \leq, 1, \times, \Rightarrow)$
  which has as objects ordered sets, as maps monotone functions,
  $f \leq g$ for $f, g : X \to Y$ iff $fx \leq^Y gx$ for all x.

- The writer/reader/state monads from FP are concurrent monads for parameters with suitable structure.

- $TX = M \times X$
  for $(M, o, +, e, \cdot)$ a concurrent monoid

- $\eta_X\, x = (o, x)$
  $\mu_X\, (m, (m', x)) = (m + m', x)$
- $\psi^0\, \star = (e, \star)$
  $\psi_{X,Y}\, ((m, x), (m', y)) = (m \cdot m', (x, y))$

- This concurrent monad is normal ($\eta_1 = \psi^0$) if the concurrent monoid is normal ($o = e$).

Reader

- $TX = S \Rightarrow X$
  for S any ordered set (it is ok if $\leq^S$ is $=$)
- NB! $S \Rightarrow X$ is the ordered set of monotone functions from S to X (if $\leq^S$ is $=$, all functions are monotone).

- $\eta_X\, x = \lambda_.\, x$
  $\mu_X\, f = \lambda s.\, f\, s\, s$
- $\psi^0\, \star = \lambda_.\, \star$
  $\psi_{X,Y}\, (f, g) = \lambda s.\, (f\, s, g\, s)$

- The interchange inequations hold as equalities, so this concurrent monad is commutative.

State

- $TX = S \Rightarrow S \times X$
  for $(S, \top, \wedge)$ a lower semilattice
  (so $\leq^S$ cannot be $=$ unless S is a singleton)
- Recall that $S \Rightarrow S \times X$ consists of monotone functions only.

- $\eta_X\, x = \lambda s.\, (s, x)$
  $\mu_X\, f = \lambda s.\, \mathsf{let}\ (s', g) = f\ s\ \mathsf{in}\ g s'$
- $\psi^0\, \star = \lambda _.\, (\top, \star)$
  $\psi_{X,Y}\, (f, g) = \lambda s.\, \mathsf{let}\ ((s_0, x), (s_1, y)) = (f\ s, g\ s)\ \mathsf{in}\ (s_0 \wedge s_1, (x, y))$

- This concurrent monad is not normal ($\eta_1 \neq \psi^0$).

- Every computation is one big atomic step.
- In parallel composition, the two given atomic steps happen "simultaneously", the competing writes are reconciled by $\wedge$.

Resumptions for interleaving shared state concurrency

- The idea of resumptions: a computation consists of small atomic steps (organized into a tree).
- $TX = \mu Z.\ \underbrace{X}_{ret} \underbrace{+}_{or} (\underbrace{S \Rightarrow}_{grab} \underbrace{\mathsf{List}}_{branch} (\underbrace{S \times}_{yield} \underbrace{Z}_{repeat}))$
  for S discretely ordered (for simplicity)
- $\leq^{TX}$ is induced by $\leq^{\mathsf{List}\,Y}$, in turn induced by order-preserving inclusion between lists

- $\eta\, x = ret\, x$
  $\mu\,(ret\, r) = r$
  $\mu\,(grab\, k) = grab\,(\lambda s.\,[(s', \mu_X\, r \mid (s', r) \leftarrow k\, s])$
- $\psi^0\, \star = ret\, \star$
  $\psi\,(ret\, x, ret\, y) = ret\,(x, y)$
  $\psi\,(ret\, x, grab\, \ell) = grab\,(\lambda s.\,[(s', \psi\,(ret\, x, r)) \mid (s', r) \leftarrow \ell\, s])$
  $\psi\,(grab\, k, ret\, y) = grab\,(\lambda s.\,[(s', \psi\,(r, ret\, y)) \mid (s', r) \leftarrow k\, s])$
  $\psi\,(grab\, k, grab\, \ell) = grab\,(\lambda s.\,[(s', \psi\,(r, grab\, \ell)) \mid (s', r) \leftarrow k\, s]$
  $+\!\!+[(s', \psi\,(grab\, k, r)) \mid (s', r) \leftarrow \ell\, s])$

- Although we use List rather than $\mathcal{M}_{\mathrm{f}}$ or $\mathcal{P}_{\mathrm{f}}$, associativity of ψ and the 4th interchange inequation hold.

Resumptions ctd.

- T supports algebraic operations for reading and writing
- $get : (S \Rightarrow X) \to TX$
  $get\ f = grab\,(\lambda s.\,[(s, f\ s)])$
- $put : S \times X \to TX$
  $put\,(s', x) = grab\,(\lambda_.\,[(s', x)])$

- T also supports a high-level control operation for atomizing computations (concatenates all small steps into one single one)
- $stitch : TX \to TX$
  $stitch\ r = grab\,(\lambda s.\,stitch'(s, r))$ where
  $stitch' : S \times TX \to \mathrm{List}\,(S \times TX)$
  $stitch'\,(s, ret\ x) = [(s, ret\ x)]$
  $stitch'\,(s, grab\ k) = concat\,(\mathrm{List}\ stitch'\,(k\ s))$

Bags of traces for interleaving shared state concurrency

- $TX = \underbrace{\mathcal{M}_{\mathrm{f}}}_{BRANCH}\ (T'X)$ where $T'X = \mu Z.\ \underbrace{X}_{ret} \underbrace{+}_{or} \underbrace{S \times}_{grab} \underbrace{S \times}_{yield} \underbrace{Z}_{repeat}$
- $\leq^{TX}$ is induced by $\leq^{\mathcal{M}_{\mathrm{f}} Y}$, induced by multiset inclusion
- $\eta\, x = \{ret\, x\}$
  $\mu\, ts = \bigcup(\mathcal{M}_{\mathrm{f}}\, \mu'\, ts)$ where
  $\mu' : T'(TX) \to TX$
  $\mu'\,(ret\, ts) = ts$
  $\mu'\,(grab\, s\, s'\, t) = T'\,(grab\, s\, s')\,(\mu'\, t)$
- $\psi^0\, \star = \{ret\, \star\}$
  $\psi\,(ts_0, ts_1) = \bigcup\{t_0 \sqcup\!\sqcup\ t_1 \mid t_0 \leftarrow ts_0, t_1 \leftarrow ts_1\}$ where
  $\sqcup\!\sqcup : T'X \times T'Y \to T(X \times Y)$
  $ret\, x \sqcup\!\sqcup ret\, y = \{ret\,(x, y)\}$
  $ret\, x \sqcup\!\sqcup grab\, s\, s'\, t = T'\,(grab\, s\, s')\,(ret\, x \sqcup\!\sqcup t)$
  $grab\, s\, s'\, t \sqcup\!\sqcup ret\, y = T'\,(grab\, s\, s')\,(t \sqcup\!\sqcup ret\, y)$
  $grab\, s_0\, s_0'\, t_0 \sqcup\!\sqcup grab\, s_1\, s_1'\, t_1 = T'\,(grab\, s_0\, s_0')\,(t_0 \sqcup\!\sqcup grab\, s_1\, s_1'\, t_1)$
  $\cup\, T'\,(grab\, s_1\, s_1')\,(grab\, s_0\, s_0'\, t_0 \sqcup\!\sqcup t_1)$
- *get*, *put*, *stitch* are also definable, e.g.,
  $put\,(s', x) = \{grab\, s\, s'\,(ret\, x) \mid s \in S\}$

Concurrent monads as concurrent monoids

- Monads on a category $\mathbb{C}$ are the same as monoids in the category $[\mathbb{C}, \mathbb{C}]$ of endofunctors on $\mathbb{C}$.

- Similarly, concurrent monads are the same as concurrent monoids in the right place.

Concurrent monoid objects

- (Classical) concurrent monoids are objects with structure in the ordered category Poset, which has finite products.

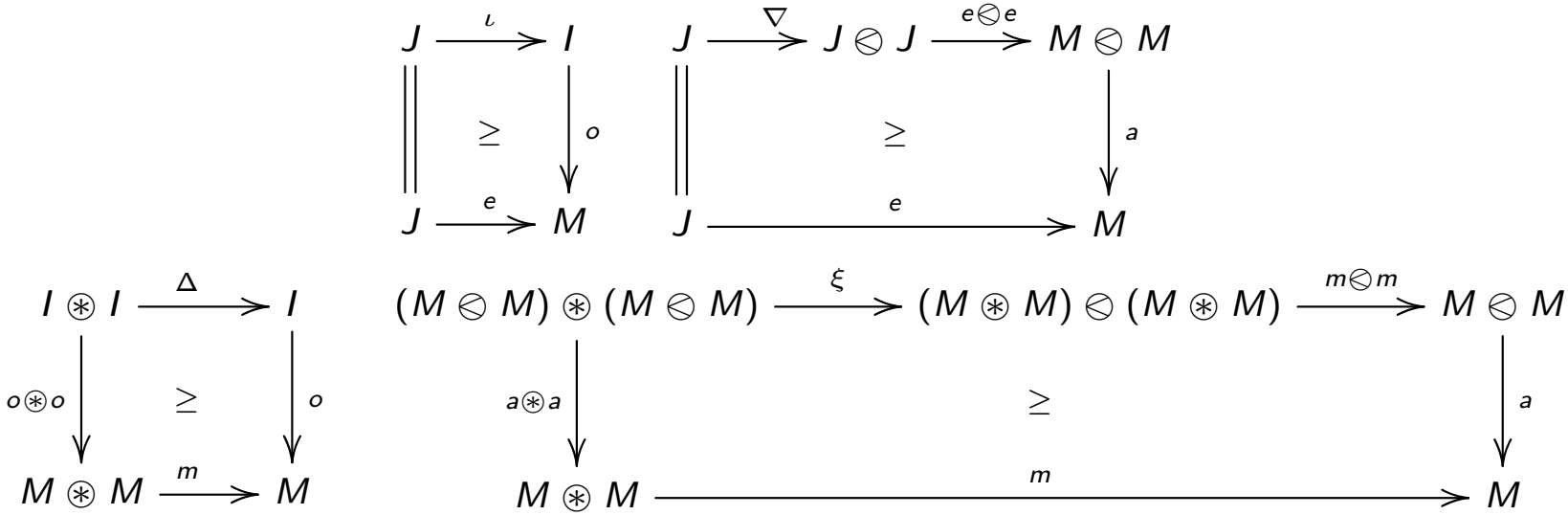
- To define what makes a concurrent monoid object in a general ordered category $\mathbb{D}$, this category has to be ordered duoidal.
- An *ordered duoidal category* is an ordered category $\mathbb{D}$ with two ordered monoidal structures $(I, \oslash)$, $(J, \circledast)$ and maps

$$\iota : J \to I$$
$$\Delta : J \to J \oslash J$$
$$\nabla : I \circledast I \to I$$
$$\xi : (F \oslash G) \circledast (H \oslash K) \to (F \circledast H) \oslash (G \circledast K)$$

natural in F, G, H, K satisfying some equations.

Concurrent monoid objects ctd

- A *concurrent monoid object* in an ordered duoidal category is an object M with two monoid structures (o, a) wrt. $(I, \oslash)$ and (e, m) wrt. $(J, \circledast)$ satisfying interchange inequations



- A (classical) concurrent monoid is a concurrent monoid object in the ordered duoidal category $(\mathrm{Poset}, 1, \times, 1, \times)$.

Concurrent monads as concurrent monoids

- If $(\mathbb{C}, \leq, \mathsf{I}, \otimes)$ is a ordered lp monoidal category, then $([\mathbb{C}, \mathbb{C}]_a, \leq, \mathsf{Id}, \cdot, \mathsf{Jd}, \star)$ is an ordered duoidal category.
- Here $(\mathsf{Jd}, \star)$ is the Day convolution monoidal structure

$$\mathsf{Jd}Z = \mathbb{C}(\mathsf{I}, Z) \bullet \mathsf{I}$$
$$(F \star G)Z = \int^{X,Y} \mathbb{C}(X \otimes Y, Z) \bullet (FX \otimes GY)$$

- An accessible concurrent monad $(T, \eta, \mu, \psi^0, \psi)$ is the same as a concurrent monoid object in this ordered duoidal category.

$$\frac{\mathsf{I} \to T\mathsf{I} \text{ in } \mathbb{C}}{\dfrac{\mathbb{C}(\mathsf{I}, Z) \to \mathbb{C}(\mathsf{I}, TZ) \text{ nat in } Z \text{ in Poset}}{\underbrace{\mathbb{C}(\mathsf{I}, Z) \bullet \mathsf{I}}_{\mathsf{Jd}Z} \to TZ \text{ nat in } Z \text{ in } \mathbb{C}}}$$

$$\frac{TX \otimes TY \to T(X \otimes Y) \text{ nat in } X, Y \text{ in } \mathbb{C}}{\dfrac{\mathbb{C}(X \otimes Y, Z) \to \mathbb{C}(TX \otimes TY, TZ) \text{ nat in } X, Y, Z \text{ in Poset}}{\dfrac{\mathbb{C}(X \otimes Y, Z) \bullet (TX \otimes TY) \to TZ \text{ nat in } X, Y, Z \text{ in } \mathbb{C}}{\underbrace{\int^{X,Y} \mathbb{C}(X \otimes Y, Z) \bullet (TX \otimes TY)}_{(T \star T)Z} \to TZ \text{ nat in } Z \text{ in } \mathbb{C}}}}$$

Concurrent monads as monads in a 2-category

- Monads (together with their underlying categories) are the same as monads in the 2-category Cat.

- Concurrent monads are the same as monads in a different 2-category.

Concurrent monads as monads in a 2-category

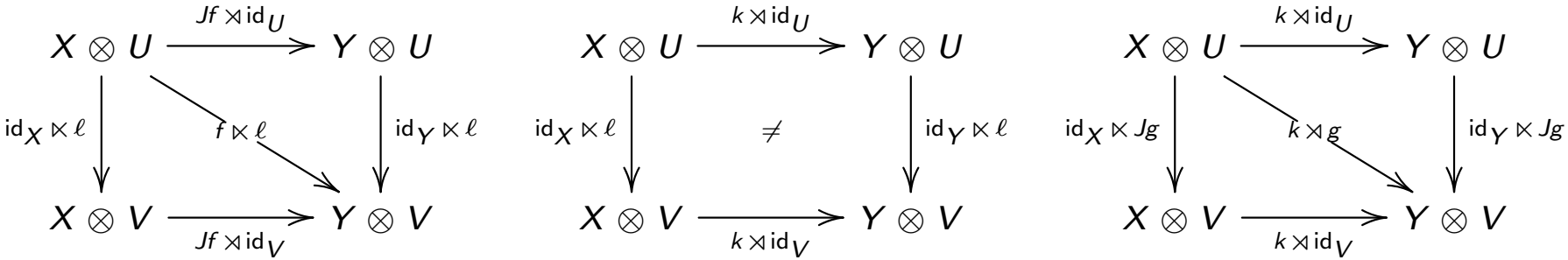
- Consider the 2-category with
  - as objects, ordered monoidal categories,
  - as 1-morphisms, ordered lax-monoidal functors,
  - as 2-morphisms, oplax-monoidal natural transformations
- Here we call a nat transf τ between ordered lax-monoidal functors $(F, \phi^0, \phi), (G, \phi^{0\prime}, \phi') : (\mathbb{C}, \mathsf{I}, \otimes) \to (\mathbb{D}, \mathsf{I}', \otimes')$ *oplax-monoidal* if

$$\begin{array}{ccc} \mathsf{I}' & \xrightarrow{\phi^0} & F\mathsf{I} \\ \| & \geq & \downarrow{\scriptstyle \tau_{\mathsf{I}}} \\ \mathsf{I}' & \xrightarrow{\phi^{0\prime}} & G\mathsf{I} \end{array} \qquad \begin{array}{ccc} FX \otimes' FY & \xrightarrow{\phi_{X,Y}} & F(X \otimes Y) \\ {\scriptstyle \tau_X \otimes' \tau_Y}\downarrow & \geq & \downarrow{\scriptstyle \tau_{X\otimes Y}} \\ GX \otimes' GY & \xrightarrow{\phi'_{X,Y}} & G(X \otimes Y) \end{array}$$

- Concurrent monads are precisely monads in this 2-category. The interchange conditions state exactly that η, μ are oplax-monoidal.
- Lax-monoidal monads (= commutative monads) are precisely monads in the 2-category where 2-morphisms are monoidal transformations, ie the squares above commute.

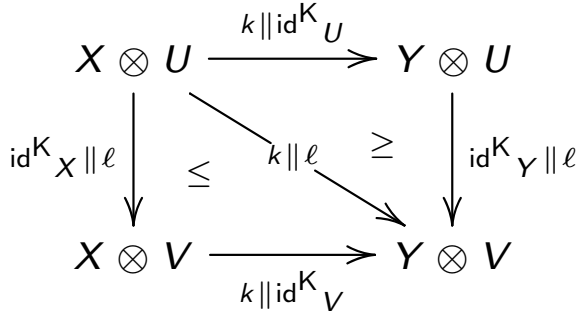
Concurrent monads vs bistrong monads

- A bistrong monad only supports parallel composition of a map with a pure map (on the left or on the right), unless it is commutative.
- In the Kleisli category of a bistrong monad (a premonoidal category) we generally have



where $\ltimes : \mathbb{C} \times \mathbb{K} \to \mathbb{K}$, $\rtimes : \mathbb{K} \times \mathbb{C} \to \mathbb{K}$.

- In the Kleisli ordered category of a concurrent monad (a concurrent category) we have



Conclusions and future work

- Parallel composition is axiomatizable relatively smoothly for typed effectful computation, but there are many subtleties.
- Ordered category theory streamlines the development.
- Resumption models for branching time can be captured well, with room for variations.

- Free constructions?
- Concurrent monads for true concurrency models, esp under branching time?
- Concurrent monads for transactional memories, relaxed memories?
- Axiomatization of atomization, combinations with other high-level control operations?

References

- J. L. Gischer. The equational theory of posets. *Theor. Comput. Sci.* 1988.
- C. A. R. Hoare, B. Möller, G. Struth, I. Wehrman. Concurrent Kleene algebra and its foundations. *J. Log. Algebraic Program.* 2011.
- E. Rivas, M. Jaskelioff. Merging monads. HAL eprint, 2019.
- V. M. Bini. Formalización de mónadas concurrentes en Agda: un análisis del caso de la mónada Delay. Lic thesis, U Nacional de Rosario, 2023.
- H. Paquet, P. Saville. Effectful semantics in bicategories: strong, commutative and concurrent pseudomonads. LICS 2024.
- E. Rivas, T. Uustalu. Concurrent monads for shared state. PPDP 2024.
- F. Breuvart, H. Paquet. Categorical continuation semantics for concurrency. FSCD 2025.
- B. Gunnarsson. Concurrent monads in 2-categories. MSc thesis, U of Iceland, 2026.
- C. Heunen, J. Sigal. Duoidally enriched Freyd categories. RAMiCS 2023.